

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)

```
import os
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

import tensorflow as tf
from tensorflow.keras import layers, regularizers
```

```
RANDOM_STATE = 42
```

```
np.random.seed(RANDOM_STATE)
random.seed(RANDOM_STATE)
tf.random.set_seed(RANDOM_STATE)
```

```
print("Random seed fixed.")
print("TensorFlow version:", tf.__version__)
```

```
Random seed fixed.
TensorFlow version: 2.20.0
```

```
file_path = "/content/drive/MyDrive/ML_Project/Group03_Task2_DataSheet.xlsx"
```

```
df = pd.read_excel(file_path)
```

```
print("Dataset loaded successfully!")
print("Shape:", df.shape)
```

```
df.head()
```

```
Dataset loaded successfully!
Shape: (70, 24)
```

	Sample No.	tanδ @ 0.000100009 Hz	tanδ @ 0.00021532 Hz	tanδ @ 0.00046404 Hz	tanδ @ 0.00100009 Hz	tanδ @ 0.0021543 Hz	tanδ @ 0.0046415 Hz	tanδ @ 0.01 Hz	tanδ @ 0.021544 Hz	tanδ @ 0.046416 Hz	...	tanδ @ 2.1546 Hz
0	1	0.313096	0.263748	0.216453	0.188339	0.194921	0.264037	0.428750	0.761844	1.390709	...	0.485241
1	2	0.308735	0.255590	0.200867	0.157484	0.132245	0.135262	0.165063	0.228676	0.358230	...	2.028017
2	3	0.307918	0.254060	0.197944	0.151697	0.120486	0.111080	0.115364	0.126730	0.148983	...	2.314646
3	4	0.571760	0.423162	0.330518	0.282384	0.273405	0.324580	0.477885	0.818407	1.469344	...	0.485744
4	5	0.565407	0.412576	0.311466	0.246326	0.203263	0.185103	0.197077	0.255297	0.391262	...	2.028166

```
5 rows × 24 columns
```

```
print(df.columns)
```

```
Index(['Sample No.', 'tanδ @ 0.000100009 Hz', 'tanδ @ 0.00021532 Hz',
      'tanδ @ 0.00046404 Hz', 'tanδ @ 0.00100009 Hz', 'tanδ @ 0.0021543 Hz',
      'tanδ @ 0.0046415 Hz', 'tanδ @ 0.01 Hz', 'tanδ @ 0.021544 Hz',
      'tanδ @ 0.046416 Hz', 'tanδ @ 0.1 Hz', 'tanδ @ 0.21544 Hz',
      'tanδ @ 0.46417 Hz', 'tanδ @ 1 Hz', 'tanδ @ 2.1546 Hz',
      'tanδ @ 4.6417 Hz', 'tanδ @ 10 Hz', 'tanδ @ 21.563 Hz',
      'tanδ @ 46.512 Hz', 'tanδ @ 100 Hz', 'tanδ @ 216.22 Hz',
      'tanδ @ 470.59 Hz', 'tanδ @ 1000 Hz', 'Class'],
      dtype='object')
```

```
# Column 1 = sample number
sample_no = df.iloc[:, 0]

# Columns 2 to 23 = 22 FDS tan(delta) features
X = df.iloc[:, 1:23].values.astype(float)

# Class column
y = df["Class"].astype(str).str.strip().values

classes = ["Dry", "Intermediate", "Wet"]
```

```
print("X shape:", X.shape)
print("y shape:", y.shape)

print("\nClass distribution:")
print(pd.Series(y).value_counts())
```

```
X shape: (70, 22)
y shape: (70,)

Class distribution:
Wet      24
Dry      23
Intermediate  23
Name: count, dtype: int64
```

```
print("Minimum value in X:", X.min())

if X.min() <= 0:
    print("Warning: X contains zero or negative values. log10 cannot be applied directly.")
else:
    print("All values are positive. log10 transformation is safe.")
```

```
Minimum value in X: 0.0024661834839625556
All values are positive. log10 transformation is safe.
```

```
X_train, X_test, y_train, y_test, sample_train, sample_test = train_test_split(
    X,
    y,
    sample_no,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=y
)
```

```
print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

```
print("\nTraining class distribution:")
print(pd.Series(y_train).value_counts())
```

```
print("\nTesting class distribution:")
print(pd.Series(y_test).value_counts())
```

```
Training samples: 56
Testing samples: 14
```

```
Training class distribution:
Intermediate    19
Wet            19
Dry            18
Name: count, dtype: int64
```

```
Testing class distribution:
Dry            5
Wet            5
Intermediate    4
Name: count, dtype: int64
```

```
X_train_log = np.log10(X_train)
X_test_log = np.log10(X_test)

print("Log transformation completed.")
```

```
Log transformation completed.
```

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train_log)
X_test_scaled = scaler.transform(X_test_log)

print("Standardization completed.")
print("Training shape:", X_train_scaled.shape)
print("Testing shape:", X_test_scaled.shape)
```

```
Standardization completed.
Training shape: (56, 22)
Testing shape: (14, 22)
```

```
label_encoder = LabelEncoder()

y_train_encoded = label_encoder.fit_transform(y_train)
y_test_encoded = label_encoder.transform(y_test)
```

```
print("Class mapping:")
for i, cls in enumerate(label_encoder.classes_):
    print(i, "->", cls)
```

```
Class mapping:
0 -> Dry
1 -> Intermediate
2 -> Wet
```

```
X_train_nn, X_val_nn, y_train_nn, y_val_nn = train_test_split(
    X_train_scaled,
    y_train_encoded,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=y_train_encoded
)

print("NN training samples:", X_train_nn.shape[0])
print("NN validation samples:", X_val_nn.shape[0])
print("NN testing samples:", X_test_scaled.shape[0])
```

```
NN training samples: 44
NN validation samples: 12
NN testing samples: 14
```

```
tf.random.set_seed(RANDOM_STATE)

nn_model = tf.keras.Sequential([
    layers.Input(shape=(22,)),

    layers.Dense(
        16,
        activation="relu",
        kernel_regularizer=regularizers.l2(0.001)
    ),
    layers.Dropout(0.20),

    layers.Dense(
        8,
        activation="relu",
        kernel_regularizer=regularizers.l2(0.001)
    ),

    layers.Dense(3, activation="softmax")
])

nn_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

nn_model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 16)	368
dropout (Dropout)	(None, 16)	0
dense_1 (Dense)	(None, 8)	136
dense_2 (Dense)	(None, 3)	27

```
Total params: 531 (2.07 KB)
Trainable params: 531 (2.07 KB)
Non-trainable params: 0 (0.00 B)
```

```
early_stop = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=25,
    restore_best_weights=True
)

history = nn_model.fit(
    X_train_nn,
    y_train_nn,
    validation_data=(X_val_nn, y_val_nn),
    epochs=300,
    batch_size=8,
```

```

callbacks=[early_stop],
verbose=1
)

```

```

Epoch 1/300
6/6 ----- 3s 88ms/step - accuracy: 0.3636 - loss: 1.1923 - val_accuracy: 0.5000 - val_loss: 1.0945
Epoch 2/300
6/6 ----- 0s 29ms/step - accuracy: 0.4318 - loss: 1.1574 - val_accuracy: 0.6667 - val_loss: 1.0639
Epoch 3/300
6/6 ----- 0s 27ms/step - accuracy: 0.4318 - loss: 1.1031 - val_accuracy: 0.6667 - val_loss: 1.0290
Epoch 4/300
6/6 ----- 0s 22ms/step - accuracy: 0.4545 - loss: 1.0662 - val_accuracy: 0.6667 - val_loss: 1.0000
Epoch 5/300
6/6 ----- 0s 24ms/step - accuracy: 0.4545 - loss: 1.0072 - val_accuracy: 0.6667 - val_loss: 0.9713
Epoch 6/300
6/6 ----- 0s 24ms/step - accuracy: 0.5227 - loss: 0.9972 - val_accuracy: 0.6667 - val_loss: 0.9447
Epoch 7/300
6/6 ----- 0s 25ms/step - accuracy: 0.4773 - loss: 0.9767 - val_accuracy: 0.5833 - val_loss: 0.9176
Epoch 8/300
6/6 ----- 0s 29ms/step - accuracy: 0.5909 - loss: 0.9130 - val_accuracy: 0.6667 - val_loss: 0.8925
Epoch 9/300
6/6 ----- 0s 34ms/step - accuracy: 0.5909 - loss: 0.9095 - val_accuracy: 0.6667 - val_loss: 0.8694
Epoch 10/300
6/6 ----- 0s 27ms/step - accuracy: 0.7273 - loss: 0.8421 - val_accuracy: 0.6667 - val_loss: 0.8480
Epoch 11/300
6/6 ----- 0s 24ms/step - accuracy: 0.7500 - loss: 0.8539 - val_accuracy: 0.6667 - val_loss: 0.8236
Epoch 12/300
6/6 ----- 0s 33ms/step - accuracy: 0.7500 - loss: 0.8452 - val_accuracy: 0.6667 - val_loss: 0.7978
Epoch 13/300
6/6 ----- 0s 33ms/step - accuracy: 0.6818 - loss: 0.8198 - val_accuracy: 0.6667 - val_loss: 0.7724
Epoch 14/300
6/6 ----- 0s 17ms/step - accuracy: 0.7273 - loss: 0.7789 - val_accuracy: 0.8333 - val_loss: 0.7485
Epoch 15/300
6/6 ----- 0s 17ms/step - accuracy: 0.7045 - loss: 0.7443 - val_accuracy: 0.8333 - val_loss: 0.7255
Epoch 16/300
6/6 ----- 0s 17ms/step - accuracy: 0.7727 - loss: 0.7542 - val_accuracy: 0.8333 - val_loss: 0.7041
Epoch 17/300
6/6 ----- 0s 17ms/step - accuracy: 0.7500 - loss: 0.7472 - val_accuracy: 0.8333 - val_loss: 0.6851
Epoch 18/300
6/6 ----- 0s 17ms/step - accuracy: 0.7727 - loss: 0.7328 - val_accuracy: 0.8333 - val_loss: 0.6684
Epoch 19/300
6/6 ----- 0s 17ms/step - accuracy: 0.7500 - loss: 0.7137 - val_accuracy: 0.8333 - val_loss: 0.6518
Epoch 20/300
6/6 ----- 0s 17ms/step - accuracy: 0.7045 - loss: 0.7056 - val_accuracy: 0.8333 - val_loss: 0.6362
Epoch 21/300
6/6 ----- 0s 20ms/step - accuracy: 0.7273 - loss: 0.6725 - val_accuracy: 0.8333 - val_loss: 0.6210
Epoch 22/300
6/6 ----- 0s 16ms/step - accuracy: 0.7500 - loss: 0.6656 - val_accuracy: 0.8333 - val_loss: 0.6070
Epoch 23/300
6/6 ----- 0s 16ms/step - accuracy: 0.7273 - loss: 0.6364 - val_accuracy: 0.9167 - val_loss: 0.5930
Epoch 24/300
6/6 ----- 0s 16ms/step - accuracy: 0.7727 - loss: 0.6272 - val_accuracy: 0.9167 - val_loss: 0.5802
Epoch 25/300
6/6 ----- 0s 16ms/step - accuracy: 0.7955 - loss: 0.6214 - val_accuracy: 0.9167 - val_loss: 0.5696
Epoch 26/300
6/6 ----- 0s 17ms/step - accuracy: 0.7045 - loss: 0.6398 - val_accuracy: 0.9167 - val_loss: 0.5608
Epoch 27/300
6/6 ----- 0s 16ms/step - accuracy: 0.7955 - loss: 0.6018 - val_accuracy: 0.9167 - val_loss: 0.5529
Epoch 28/300
6/6 ----- 0s 16ms/step - accuracy: 0.7273 - loss: 0.6233 - val_accuracy: 0.8333 - val_loss: 0.5473
Epoch 29/300
6/6 ----- 0s 16ms/step - accuracy: 0.8182 - loss: 0.5785 - val_accuracy: 0.8333 - val_loss: 0.5419

```

```

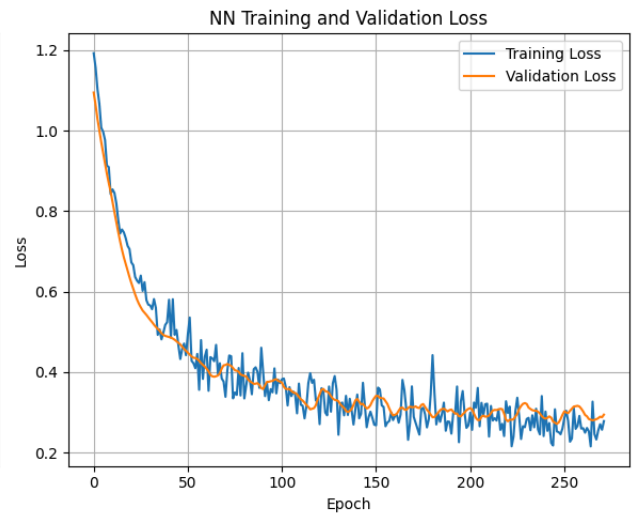
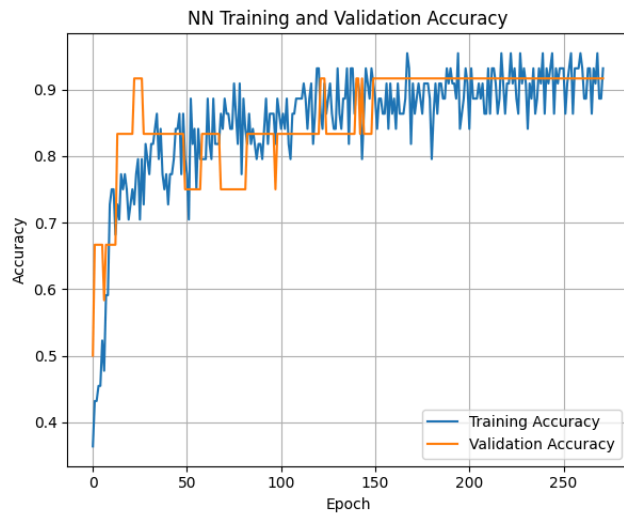
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("NN Training and Validation Accuracy")
plt.grid(True)
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("NN Training and Validation Loss")
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()

```



```
train_loss, train_acc = nn_model.evaluate(X_train_nn, y_train_nn, verbose=0)
val_loss, val_acc = nn_model.evaluate(X_val_nn, y_val_nn, verbose=0)
test_loss, test_acc = nn_model.evaluate(X_test_scaled, y_test_encoded, verbose=0)

print(f"NN Training Accuracy : {train_acc * 100:.2f}%")
print(f"NN Validation Accuracy : {val_acc * 100:.2f}%")
print(f"NN Testing Accuracy : {test_acc * 100:.2f}%")

print(f"\nNN Training Loss : {train_loss:.4f}")
print(f"NN Validation Loss : {val_loss:.4f}")
print(f"NN Testing Loss : {test_loss:.4f}")
```

```
NN Training Accuracy : 93.18%
NN Validation Accuracy : 91.67%
NN Testing Accuracy : 85.71%
```

```
NN Training Loss : 0.2305
NN Validation Loss : 0.2713
NN Testing Loss : 0.4545
```

```
y_test_prob_nn = nn_model.predict(X_test_scaled)
y_test_pred_encoded = np.argmax(y_test_prob_nn, axis=1)

y_test_pred_nn = label_encoder.inverse_transform(y_test_pred_encoded)

prediction_df = pd.DataFrame({
    "Sample_No": sample_test.values,
    "Actual_Class": y_test,
    "Predicted_Class": y_test_pred_nn
})

prediction_df
```

1/1 ————— 0s 194ms/step

	Sample_No	Actual_Class	Predicted_Class
0	46	Intermediate	Wet
1	15	Dry	Dry
2	45	Intermediate	Intermediate
3	35	Intermediate	Intermediate
4	48	Wet	Wet
5	40	Intermediate	Intermediate
6	8	Dry	Dry
7	55	Wet	Wet
8	60	Wet	Wet
9	64	Wet	Wet
10	20	Dry	Dry
11	50	Wet	Wet
12	7	Dry	Dry
13	11	Dry	Intermediate

```
print("Neural Network Classification Report:")
print(classification_report(y_test, y_test_pred_nn, labels=classes))
```

```
Neural Network Classification Report:
              precision    recall  f1-score   support

   Dry               1.00      0.80      0.89         5
 Intermediate        0.75      0.75      0.75         4
    Wet              0.83      1.00      0.91         5

 accuracy                   0.86         14
 macro avg              0.86      0.85      0.85         14
 weighted avg           0.87      0.86      0.86         14
```

```
cm_nn = confusion_matrix(y_test, y_test_pred_nn, labels=classes)

plt.figure(figsize=(6, 5))
plt.imshow(cm_nn, cmap="Blues")

plt.title("Neural Network Confusion Matrix")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")

plt.xticks(np.arange(len(classes)), classes)
plt.yticks(np.arange(len(classes)), classes)

for i in range(len(classes)):
    for j in range(len(classes)):
        plt.text(j, i, cm_nn[i, j], ha="center", va="center", fontsize=12)

plt.colorbar()
plt.tight_layout()
plt.show()
```

